学案 01 | 文本、Tokenizer 与 next-token 数据

1. tokenizer 解决什么问题

神经网络不能直接接收"你好"或 `hello` 这样的字符串。Tokenizer 把文本转换成有限词表中的 token，再把每个 token 映射为整数 ID：

```
文本 → token 序列 → token ID 序列 → embedding 查表 → 模型输入
```

token 不一定等于"一个字"或"一个词"。常见粒度有：

- **字符级**：实现简单、词表小，但序列长；
- **词级**：直观，但未知词和词形变化会导致词表膨胀；
- **子词级**：在词表大小与序列长度之间折中，是现代 LLM 常用路线；
- **字节级**：先把任意文本表示成字节，避免 Unicode 字符集合固定不全，但需要学习字节到 token 的合并方式。

本阶段用字符级 tokenizer 观察机制，不把它当作生产 tokenizer。

2. 最小字符 tokenizer

给定训练文本，先收集其中出现过的不同字符并排序：

```
词表 = [' ', 'a', 'b', 'c', ...]
stoi: 字符 → ID
itos: ID → 字符
```

编码时查 `stoi`，解码时查 `itos`。例如词表中 `a=1,b=2`，文本 `aba` 编码为 `[1,2,1]`。

它的关键不在字典语法，而在三个不变量：

1. `decode(encode(text)) == text`（文本只包含已知 token 时）；
2. 每个 ID 都在 `[0, vocab_size)`；
3. 模型的 embedding 行数必须等于 `vocab_size`。

生产 tokenizer 还需要处理未知字符、特殊 token、规范化、空格策略、训练语料与版本固定。Tokenizer 版本变化会改变 ID，因而不能随意替换。

3. 为什么训练样本要错开一位

语言模型学习的是：给定此前 token，预测下一个 token。对 ID 序列：

```
ids = [a, b, c, d, e]
x   = [a, b, c, d]
y   = [b, c, d, e]
```

每个位置的 `x[t]` 是上下文，`y[t]` 是它之后应出现的 token。模型输出 `logits[:, t, :]`，和 `y[:, t]` 比较。

若批次形状是：

```
x:      (B, T)
y:      (B, T)
logits: (B, T, V)
```

其中 `B` 是 batch size，`T` 是上下文长度，`V` 是词表大小。交叉熵通常把 logits 变成 `(B*T, V)`，把目标变成 `(B*T,)`：

```
loss = cross_entropy(logits.reshape(B * T, V), y.reshape(B * T))
```

这里的 reshape 是把位置摊平，不是改变 token 的对应关系。

4. 训练/验证切分与滑动窗口

不能把同一段文本的未来内容泄露给训练输入。最小实验中可先按文本位置切分 train/val，再从各自区域随机取连续片段：

```
train: ├──────────────────────┤
val:                         ├──────────┤
```

窗口 `x[start:start+T]` 与 `y[start+1:start+T+1]` 必须来自同一分区。`start+T+1` 超出分区时应重新采样或缩短窗口。

5. 子词 tokenizer 的直觉

BPE 类方法从字符或字节开始，反复合并语料中常见的相邻片段。常见片段会成为一个 token，罕见词仍可拆成更小片段。SentencePiece 的重要工程思想是：可以直接在原始文本上训练，并把空格等边界纳入可逆表示，而不是强依赖语言特定的预分词器。

学习时要区分：

- tokenizer 的**分段规则**；
- token 到 ID 的**编号**；
- ID 到 embedding 的**参数查表**。

Tokenizer 不理解事实真假，也不负责生成文本；它只规定字符串与离散 ID 的接口。

6. 运行与观察

运行：

```
& '.\.venv\Scripts\python.exe' '.\阶段1_decoder_only生成与nanoGPT\代码
```

```
\01_toy_tokenizer.py'
```


重点记录：词表、编码结果、解码结果、`x/y` 的长度和每个位置的预测目标。然后修改示例文本，观察词表大小和未知字符错误如何变化。

7. 常见错误

- 把 token 当成固定的“词”；实际粒度由 tokenizer 决定。
- `x` 与 `y` 没有错开，模型等于在复制当前 token，训练目标就变了。
- 把目标 ID 转成浮点数；`CrossEntropyLoss` 的类别目标通常需要整数类型。
- 在验证集上重新建立独立词表；这会使 ID 空间不一致。通常 tokenizer 由训练语料建立，再用于验证/测试。
- 忘记保存 tokenizer 配置和词表；模型权重与 token ID 解释会失配。
- 只看一条 decode 结果就断言 tokenizer 正确；至少检查 round-trip 和边界字符。

8. 小练习

1. 对 `hello` 构造字符级词表，手算 `encode` 与 `decode`。
2. 给出 `ids=[4,7,2,9]`、`T=3` 时的 `x` 和 `y`。
3. 如果 `V=10`，某位置的 logits 形状为什么是 `(10,)`，而不是一个 token ID?
4. 修改 `代码/01_toy_tokenizer.py`，加入一个显式的 `<unk>`，并说明它解决了什么、没有解决什么。